Grade 2

IT Department
Web Development

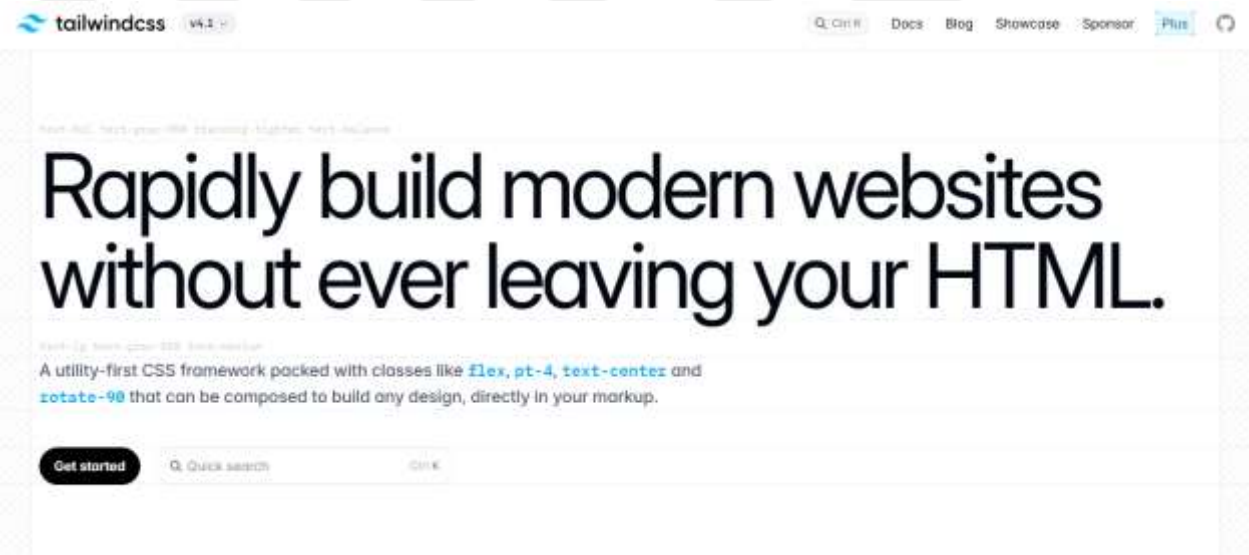
On-site 4

ON-SITE GUIDE
2025 - 2026

Tailwind CSS - Theoretical Lesson -Part 1

1. Introduction to tailwind CSS:

Tailwind CSS is a modern, utility-first class framework that allows developers to style their websites directly within HTML using concise utility classes. Unlike traditional CSS, Tailwind CSS promotes rapid development by eliminating the need to write custom styles for every component.



.

Tailwind CSS works by scanning all of your HTML files, JavaScript components, and any other templates for class names, generating the corresponding styles and then writing them to a static CSS file.

It's fast, flexible, and reliable — with zero-runtime.

1.1 Why Choose Tailwind Over Other CSS Frameworks?

Tailwind CSS stands out from traditional frameworks like Bootstrap or Foundation because of its **utility-first methodology**, which offers:

- **Full control of small design details**: Tailwind provides control at the atomic level, allowing you to customize each aspect of your design.
- **Faster prototyping**: By using utility classes, developers can iterate faster without worrying about conflicting styles or overriding pre-built components.

- **Lightweight code**: Tailwind generates smaller CSS files by purging unused styles, improving website performance with faster load times.
- **Simplified responsive design**: Tailwind's utility classes make responsive design effortless without the need for custom media queries.

2. Installation

Depend on The last Update of Tailwind CSS V4.1 you can install tailwind depending on The Type of Project or The Frame work Used , So You can install using:

1-Vite

2-Post CSS

3- Tailwind CLI

4-CDN

5-Framwork You Work with

The simplest and fastest way to get up and running with Tailwind CSS from scratch is with the Tailwind CLI tool and we will use to create our Projects.

2.1 Install Tailwind CSS Using Tailwind CLI:

➢ **Install Tailwind CSS:** Install tailwindcss and @tailwindcss/cli via npm , open vs code terminal and type this command.

```
npm install tailwindcss @tailwindcss/cli
```

After installing tailwind a node_modules folder, package.json and package-lock.json are downloaded.

➢ **Import Tailwind in your CSS**

In your project create a folder called src inside it create a folder called called input.css , in this file Put this Code to import tailwind.

```
@import "tailwindcss";
```

- **Start the Tailwind CLI build process:**

  Run the CLI tool to scan your source files for classes and build your CSS , this will create a file called output.css in the src folder containing all tailwind classes you use and converting it into css Rules.

```
npx @tailwindcss/cli -i ./src/input.css -o ./src/output.css --watch
```

--watch Flag: tells the Tailwind CLI to **keep running in the background** and **automatically recompile your CSS file** (output.css) **every time you make changes** to the input file (input.css) or your HTML/JS files that use Tailwind classes.

- **Start using Tailwind in your HTML:**

Add your compiled CSS file to the <head> and start using Tailwind's utility classes to style your content.

Activtiy:

Use All Steps To create A simple WE SCHOOL Heading Using Tailwind :

```html
<!doctype html>
<html>
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link href="./output.css" rel="stylesheet">
</head>
<body>
  <h1 class="text-3xl font-bold text-center bg-purple-700 text-white p-5">
    WE SCHOOL
  </h1>
</body>
</html>
```

Output:

WE SCHOOL

3. Tailwind CSS IntelliSense

To improve your experience using Tailwind CSS, especially in editors like VS Code, it's recommended to install the official Tailwind CSS IntelliSense extension. It provides helpful features like autocomplete, syntax highlighting, and linting. Since Tailwind uses special CSS syntax (@theme, @variant), in some editors to avoid warnings you may also need to disable strict CSS highlighting errors and potential bugs in both your CSS and your markup.

4. Colors In Tailwind CSS:

Using and customizing the color palette in Tailwind CSS projects.

Tailwind CSS includes a vast, beautiful color palette out of the box, carefully crafted by expert designers and suitable for a wide range of different design styles, Every color in the default palette includes 11 steps, with 50 being the lightest, and 950 being the darkest:

The entire color palette is available across all color related utilities, including things like background color, border color, fill, caret color, and many more.

```
<div>
  <div class="bg-sky-50"></div>
  <div class="bg-sky-100"></div>
  <div class="bg-sky-200"></div>
  <div class="bg-sky-300"></div>
  <div class="bg-sky-400"></div>
  <div class="bg-sky-500"></div>
  <div class="bg-sky-600"></div>
  <div class="bg-sky-700"></div>
  <div class="bg-sky-800"></div>
  <div class="bg-sky-900"></div>
  <div class="bg-sky-950"></div>
</div>
```

5. Backgrounds In Tailwind CSS:

5.1 background-attachment:

Utilities for controlling how a background image behaves when scrolling.

Class	Styles
bg-fixed	background-attachment: fixed;
bg-local	background-attachment: local;
bg-scroll	background-attachment: scroll;

- Use the bg-fixed utility to fix the background image relative to the viewport.
- Use the bg-local utility to scroll the background image with the container and the viewport.
- Use the bg-scroll utility to scroll the background image with the viewport, but not with the container.

5.2 background-color:

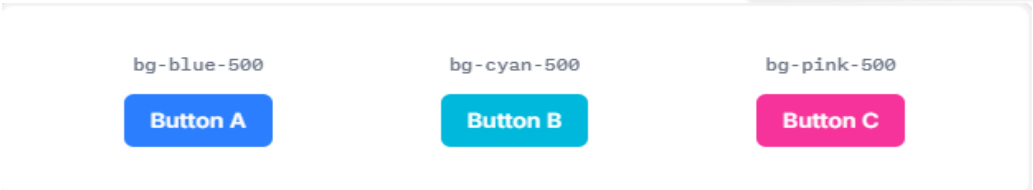
Utilities for controlling an element's background color.

Class	Styles
bg-inherit	background-color: inherit;
bg-current	background-color: currentColor;
bg-transparent	background-color: transparent;
bg-black	background-color: var(--color-black); /* *#000* */
bg-white	background-color: var(--color-white); /* *#fff* */
bg-{color}-{shade}	background-color: var(--color- color - shade);

Example:

```
<button class="bg-blue-500">Button A</button>
<button class="bg-cyan-500">Button B</button>
<button class="bg-pink-500">Button C</button>
```

Output:

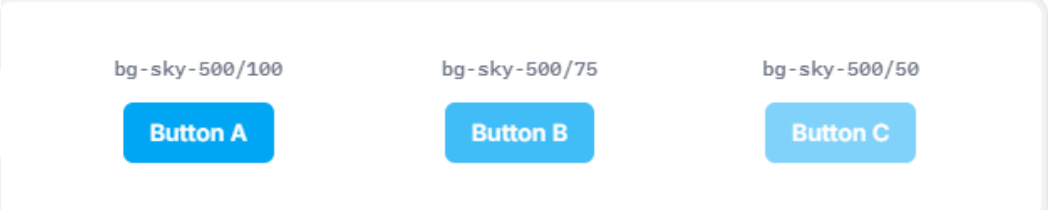


5.3 Changing the opacity

Use the color opacity modifier to control the opacity of an element's background color:

```html
<button class="bg-sky-500/100"></button>
<button class="bg-sky-500/75"></button>
<button class="bg-sky-500/50"></button>
```

Output:



5.4 Using a custom value:

Use the bg-[<*value*>] syntax to set the background color based on a completely custom value:

```html
<div class="bg-[#50d71e] ...">
  <!-- ... -->
</div>
```

5.5 background-image:

Utilities for controlling an element's background image.

Class	Styles
bg-[<*value*>]	background-image: <*value*>;
bg-(image:<*custom-property*>)	background-image: var(<*custom-property*>);
bg-none	background-image: none;
bg-linear-to-{direction}	background-image: linear-gradient(to { direction}, var(--tw-gradient-stops));

Available Directions To make Gradient Background:

Direction Class	Meaning
bg-gradient-to-t	Bottom → Top
bg-gradient-to-b	Top → Bottom
bg-gradient-to-l	Right → Left
bg-gradient-to-r	Left → Right
bg-gradient-to-tr	Bottom Left → Top Right
bg-gradient-to-br	Top Left → Bottom Right
bg-gradient-to-bl	Top Right → Bottom Left
bg-gradient-to-tl	Bottom Right → Top Left

Examples:

se the bg-[<*value*>] syntax to set the background image of an element:

```
<div class="bg-[url(/img/mountains.jpg)] ..."></div>
```

Adding a linear gradient

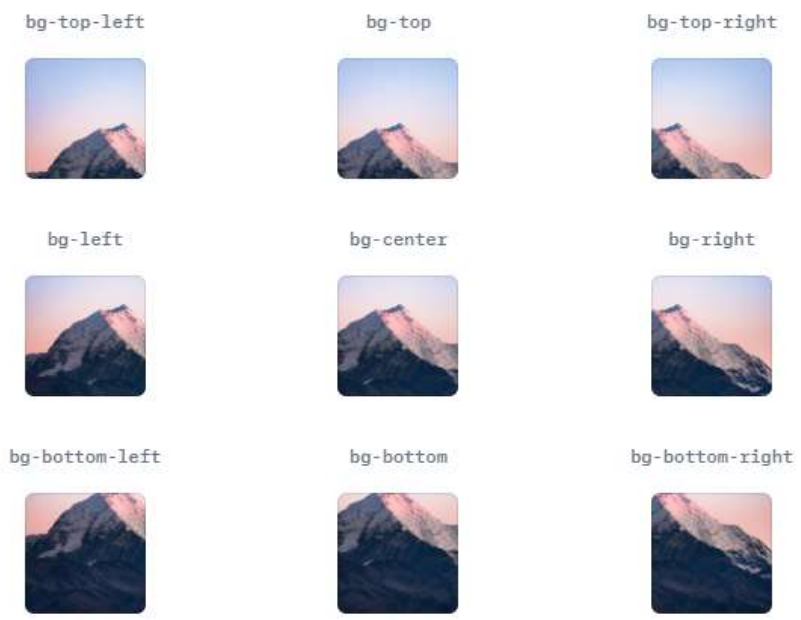
Use utilities like bg-linear-to-r and bg-linear-<*angle*> with the **color stop utilities** to add a linear gradient to an element:

```
<div class="h-14 bg-linear-to-r from-cyan-500 to-blue-500"></div>
<div class="h-14 bg-linear-to-t from-sky-500 to-indigo-500"></div>
<div class="h-14 bg-linear-to-bl from-violet-500 to-fuchsia-500"></div>
<div class="h-14 bg-linear-65 from-purple-500 to-pink-500"></div>
```

5.6 background-position

Utilities for controlling the position of an element's background image.

Class	Styles
bg-top-left	background-position: top left;
bg-top	background-position: top;
bg-top-right	background-position: top right;
bg-left	background-position: left;
bg-center	background-position: center;
bg-right	background-position: right;
bg-bottom-left	background-position: bottom left;
bg-bottom	background-position: bottom;
bg-bottom-right	background-position: bottom right;



5.7 background-repeat

Utilities for controlling the repetition of an element's background image.

Class	Styles
bg-repeat	background-repeat: repeat;
bg-repeat-x	background-repeat: repeat-x;
bg-repeat-y	background-repeat: repeat-y;
bg-no-repeat	background-repeat: no-repeat;

5.8 background-size:

Utilities for controlling the background size of an element's background image.

Class	Styles
bg-auto	background-size: auto;
bg-cover	background-size: cover;
bg-contain	background-size: contain;

Example:

```
1  <div class="bg-[url(/img/mountains.jpg)] bg-cover bg-center"></div>
```

6. Hover, focus, and other states In Tailwind CSS:

Using utilities to style elements on hover, focus, and more.

Every utility class in Tailwind can be applied *conditionally* by adding a variant to the beginning of the class name that describes the condition you want to target.

For example, to apply the bg-sky-700 class on hover, use the hover:bg-sky-700 class:

```
1  <button class="bg-sky-500 hover:bg-sky-700 ...">Save changes</button>
```

Save changes

Hover over this button to see the background color change:

Save changes

Tailwind includes variants for just about everything you'll ever need, including:

- **Pseudo-classes**, like :hover, :focus, :first-child .
- **Pseudo-elements**, like ::before, ::after .
- **Media and feature queries**, like responsive breakpoints, dark mode, and prefers-reduced-motion
- **Attribute selectors**, like [dir="rtl"] .
- **Child selectors**, like & > * and & *

These variants can even be stacked to target more specific situations.

:hover, :focus, and :active

Style elements on hover, focus, and active using the hover, focus, and active variants:

```
<button class="bg-violet-500 hover:bg-violet-600 focus:outline-2 focus:outline-offset-2 focus:outline-violet-500 active:bg-violet-700 ...">
  Save changes
</button>
```

The difference here in this example when we press on the button focus class work and make outline with violet color:

Save changes

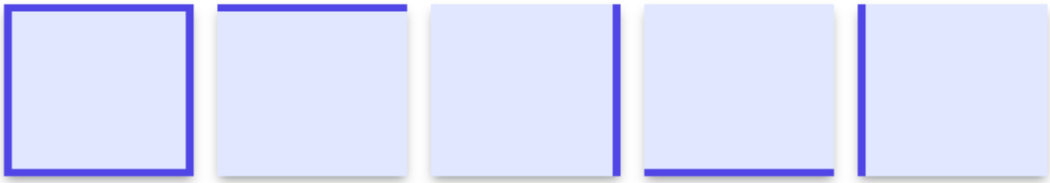
7. Borders In Tailwind CSS:

To add borders using Tailwind CSS, you can use a combination of utility classes that control the style, width, color, and rounding of the border:

- Firstly, Decide which html element you want to add the border to. It could be a , <button>, <input>, or any other html element.

- Apply border utility classes from Tailwind CSS with border properties if you want to add any.

```
<p class="w-24 h-24 border border-solid border-indigo-600"></p>
<p class="w-24 h-24 border-t border-solid border-indigo-600 "></p>
<p class="w-24 h-24 border-r border-solid border-indigo-600 "></p>
<p class="w-24 h-24 border-b border-solid border-indigo-600 "></p>
<p class="w-24 h-24 border-l border-solid border-indigo-600 "></p>
```



7.1 Border Color:

Change the border color using utilities built on our theme colors ,Use below tailwind border color example to change color of border.

```
<p class="w-24 h-24 border-4 border-solid border-indigo-600 bg-indigo-100 "></p>
<p class="w-24 h-24 border-4 border-solid border-gray-600 bg-gray-100 "></p>
<p class="w-24 h-24 border-4 border-solid border-red-600 bg-red-100"></p>
<p class="w-24 h-24 border-4 border-solid border-yellow-600 bg-yellow-100 "></p>
<p class="w-24 h-24 border-4 border-solid border-green-600 bg-green-100 "></p>
```

7.2 Border radius:

Add utility classes to an element to easily round its corners, In following exmaple of tailwind border radius shows how to give border radius in tailwind css.

7.3 Border radius Sizes:

Use following tailwind rounded border to easily round corners of any elements.

7.4 Border Style:

Border Style classes are used to style borders in tailwind css border.

8. Position In Tailwind CSS:

Utilities for controlling how an element is positioned in the document.

Class	Styles
static	position: static;
fixed	position: fixed;
absolute	position: absolute;
relative	position: relative;
sticky	position: sticky;

8.1 top / right / bottom / left Utilities:

Utilities for controlling the placement of positioned elements.

Use top-<*number*>, right-<*number*>, bottom-<*number*>, left-<*number*>, and inset-<*number*> utilities like top-0 and bottom-4 to set the horizontal or vertical position of a positioned element:

8.2 z-index In Tailwind CSS:

Utilities for controlling the stack order of an element.

Class	Styles
z-<*number*>	z-index: <*number*>;
z-auto	z-index: auto;
z-[<*value*>]	z-index: <*value*>;

Use the z-<*number*> utilities like z-10 and z-50 to control the stack order (or three-dimensional positioning) of an element, regardless of the order it has been displayed:

```
<div class="z-40">05</div>
<div class="z-30">04</div>
<div class="z-20">03</div>
<div class="z-10">02</div>
<div class="z-0">01</div>
```

9. Responsive design

Using responsive utility variants to build adaptive user interfaces.

There are five breakpoints by default, inspired by common device resolutions:

Breakpoint prefix	Minimum width	CSS
Sm	40rem *(640px)*	@media (width >= 40rem) { ... }
Md	48rem *(768px)*	@media (width >= 48rem) { ... }
Lg	64rem *(1024px)*	@media (width >= 64rem) { ... }
Xl	80rem *(1280px)*	@media (width >= 80rem) { ... }
2xl	96rem *(1536px)*	@media (width >= 96rem) { ... }

This works for **every utility class in the framework**, which means you can change literally anything at a given breakpoint — even things like letter spacing or cursor styles.

9.1 Working mobile-first:

Tailwind uses a mobile-first breakpoint system, similar to what you might be used to in other frameworks like Bootstrap.

What this means is that unprefixed utilities (like text-white) take effect on all screen sizes, while prefixed utilities (like md: text-white) only take effect at the specified breakpoint *and above*.

Tailwind CSS - Theoretical Lesson -Part 2

1.1 font-family:

Utilities for controlling the font family of an element.

Class	Styles
font-sans	font-family: var(--font-sans); /* *ui-sans-serif, system-ui, sans-serif, 'Apple Color Emoji', 'Segoe UI Emoji', 'Segoe UI Symbol', 'Noto Color Emoji'* */
font-serif	font-family: var(--font-serif); /* *ui-serif, Georgia, Cambria, 'Times New Roman', Times, serif* */
font-mono	font-family: var(--font-mono); /* *ui-monospace, SFMono-Regular, Menlo, Monaco, Consolas, 'Liberation Mono', 'Courier New', monospace* */
font-[<*value*>]	font-family: <*value*>;

Example:

```
1  <p class="font-sans">The quick brown fox</p>
2  <p class="font-serif">The quick brown fox</p>
3  <p class="font-mono">The quick brown fox</p>
```

```
font-sans
The quick brown fox jumps over the lazy dog.

font-serif
The quick brown fox jumps over the lazy dog.

font-mono
The quick brown fox jumps over the lazy dog.
```

1.2 **Using a custom value**:

Use the font-[<*value*>] syntax to set the font family based on a completely custom value:

```
<p class="font-[Open_Sans]">
  Lorem ipsum dolor sit amet...
</p>
```

1.3 font-size:

Utilities for controlling the font size of an element.

Class	Font Size
text-xs	0.75rem (12px)
text-sm	0.875rem (14px)
text-base	1rem (16px)
text-lg	1.125rem (18px)
text-xl	1.25rem (20px)
text-2xl	1.5rem (24px)
text-3xl	1.875rem (30px)
text-4xl	2.25rem (36px)
text-5xl	3rem (48px)
text-6xl	3.75rem (60px)
text-7xl	4.5rem (72px)
text-8xl	6rem (96px)
text-9xl	8rem (128px)

```
<p class="text-sm ">The quick brown fox </p>
<p class="text-base ">The quick brown fox </p>
<p class="text-lg ">The quick brown fox </p>
<p class="text-xl ">The quick brown fox </p>
<p class="text-2xl ">The quick brown fox </p>
```

1.4 font-weight:

Utilities for controlling the font weight of an element.

Class	Styles
font-thin	font-weight: 100;
font-extralight	font-weight: 200;
font-light	font-weight: 300;
font-normal	font-weight: 400;
font-medium	font-weight: 500;
font-semibold	font-weight: 600;
font-bold	font-weight: 700;
font-extrabold	font-weight: 800;
font-black	font-weight: 900;

Example:

```
<p class="font-normal ">The quick brown fox </p>
<p class="font-medium ">The quick brown fox </p>
<p class="font-semibold ">The quick brown fox ...</p>
```

1.5 font-style:

Utilities for controlling the style of text.

Class	Styles
Italic	font-style: italic;
not-italic	font-style: normal;

1.6 text-align:

Utilities for controlling the alignment of text.

Class	Styles
text-left	text-align: left;
text-center	text-align: center;
text-right	text-align: right;
text-justify	text-align: justify;
text-start	text-align: start;
text-end	text-align: end;

2. Spacing Utilities In Tailwind CSS:

2.1 padding:

Utilities for controlling an element's padding.

Class	Styles
p-<*number*>	padding: calc(var(--spacing) * <*number*>);
p-px	padding: 1px;
p-[<*value*>]	padding: <*value*>;
px-<*number*>	padding-inline: calc(var(--spacing) * <*number*>);
py-<*number*>	padding-block: calc(var(--spacing) * <*number*>);
pt-<*number*>	padding-top: calc(var(--spacing) * <*number*>);
pb-<*number*>	padding-bottom: calc(var(--spacing) * <*number*>);
pl-<*number*>	padding-left: calc(var(--spacing) * <*number*>);
pr--<*number*>	padding-right: calc(var(--spacing) * <*number*>);

```
1   <div class="p-8 ...">p-8</div>
```

```
<div class="pt-6 ">pt-6</div>
<div class="pr-4 ">pr-4</div>
<div class="pb-8 ">pb-8</div>
<div class="pl-2 ">pl-2</div>
```

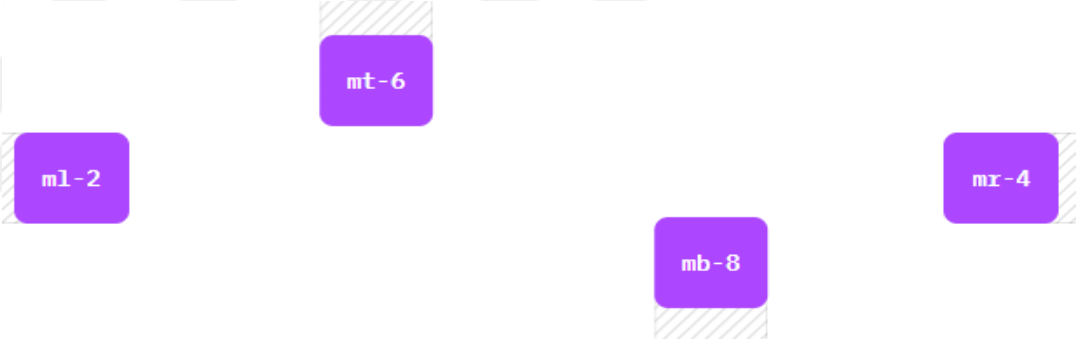
2.2 margin:

Utilities for controlling an element's margin.

Class	Styles
m-<*number*>	margin: calc(var(--spacing) * <*number*>);
-m-<*number*>	margin: calc(var(--spacing) * -<*number*>);
m-auto	margin: auto;
m-px	margin: 1px;
-m-px	margin: -1px;
m-[<*value*>]	margin: <*value*>;
mx-<*number*>	margin-inline: calc(var(--spacing) * <*number*>);
mx-auto	margin-inline: auto;
my-<*number*>	margin-block: calc(var(--spacing) * <*number*>);
my-auto	margin-block: auto;
mt-<*number*>	margin-top: calc(var(--spacing) * <*number*>);
mr-<*number*>	margin-right: calc(var(--spacing) * <*number*>);
mb-<*number*>	margin-bottom: calc(var(--spacing) * <*number*>);
ml-<*number*>	margin-left: calc(var(--spacing) * <*number*>);

```
<div class="m-8 ...">m-8</div>
```

```
<div class="mt-6">mt-6</div>
<div class="mr-4">mr-4</div>
<div class="mb-8">mb-8</div>
<div class="ml-2">ml-2</div>
```



In **v4 (including v4.1)**, the **default maximum value** for padding and margin utilities is: p-64, m-64.

3. Sizing In Tailwind CSS:

3.1 width:

Utilities for setting the width of an element.

Tailwind provides both fixed, fractional, and percentage-based, width utilities.

3.2 Fixed Widths:

Use w-<*number*> utilities like w-24 and w-64 to set an element to a fixed width based on the spacing scale:

Class	Width Value	Class	Width Value

w-0	0rem (0px)	w-16	4rem (64px)
w-1	0.25rem (4px)	w-32	8rem (128px)
w-2	0.5rem (8px)	w-64	16rem (256px)
w-4	1rem (16px)	w-px	1px
w-8	2rem (32px)		

3.3 Percentage Widths:

Use w-full or w-<*fraction*> utilities like w-1/2 and w-2/5 to give an element a percentage-based width:

Class	Width
w-full	100%
w-1/2	50%
w-1/3	33.3333%
w-2/3	66.6667%
w-1/4	25%
w-3/4	75%
w-1/5 to w-5/6	Fractions up to 83.3333%

3.3 Viewport-Based Widths:

Class	Width
w-screen	100vw (full screen width)
w-[50vw]	Arbitrary viewport width

3.4 containers width :

Use utilities like w-sm and w-xl to set an element to a fixed width based on the container scale:

Class	CSS Variable	Width Value	Pixels	Class	CSS Variable	Width Value	Pixels
w-3xs	--container-3xs	16rem	256px	w-2xl	--container-2xl	42rem	672px
w-2xs	--container-2xs	18rem	288px	w-3xl	--container-3xl	48rem	768px
w-xs	--container-xs	20rem	320px	w-4xl	--container-4xl	56rem	896px
w-sm	--container-sm	24rem	384px	w-5xl	--container-5xl	64rem	1024px
w-md	--container-md	28rem	448px	w-6xl	--container-6xl	72rem	1152px
w-lg	--container-lg	32rem	1024px	w-7xl	--container-7xl	80rem	1280px
w-xl	--container-xl	36rem	576px				

4. Height In Tailwind CSS:

Utilities for setting the height of an element.

4.1 Fixed Height (Spacing Scale):

Class	Height Value	Class	Height Value
h-0	0rem (0px)	h-8	2rem (32px)
h-1	0.25rem (4px)	h-16	4rem (64px)
h-2	0.5rem (8px)	h-32	8rem (128px)
h-4	1rem (16px)	h-64	16rem (256px)
h-px	1px		

4.2 Percentage Heights:

Class	Height Value	Class	Height Value

h-full	100%	h-2/3	66.6667%
h-1/2	50%	h-1/4	25%
h-1/3	33.3333%	h-3/4	75%

4.3 Screen-Based Heights:

Class	Height Value
h-screen	100vh (full screen height)
h-[50vh]	Arbitrary screen height

5. Tailwind CSS Flexbox Utilities:

5.1 Display Types:

Class	Description
Flex	Enables Flexbox
inline-flex	Flex layout inline

5.2 Flex Direction

Class	Direction
flex-row	Horizontal (default)
flex-row-reverse	Reversed horizontal
flex-col	Vertical
flex-col-reverse	Reversed vertical

5.3 Flex Wrap

Class	Behavior
flex-wrap	Allows wrapping
flex-nowrap	Prevents wrapping
flex-wrap-reverse	Reverse wrap

5.4 Align Items (align-items)

Class	Behavior
items-start	Align to top/start
items-center	Vertically centered
items-end	Align to bottom/end
items-stretch	Stretch to fill (default)

5.5 Justify Content

Class	Behavior
justify-start	Left/top aligned
justify-center	Centered
justify-end	Right/bottom aligned

justify-between	Space between
justify-around	Space around
justify-evenly	Equal space between items

5.6 Align Self (Per Item)

Class	Effect on Individual Item
self-start	Align to start
self-center	Centered
self-end	Align to end
self-stretch	Fill available space

5.7 Flex Grow/Shrink

Class	Behavior
flex-1	Grow and shrink (equal)
flex-auto	Grow/shrink as needed
flex-initial	Initial size only
flex-none	No growth/shrink

6. Tailwind CSS Grid Utilities

6.1 Display Type:

Class	Description
Grid	Enables grid layout
inline-grid	Inline grid

6.2 Grid Columns:

Class	Description
grid-cols-1 to grid-cols-12	Set number of columns
grid-cols-none	Remove grid columns

6.3 Grid Rows:

Class	Description
grid-rows-1 to grid-rows-6	Number of rows
grid-rows-none	No row definition

6.4 Gap Between Items

Class	Description
gap-4	Gap between all rows/columns
gap-x-4	Horizontal gap only
gap-y-4	Vertical gap only

6.5 Item Placement

Class	Description
place-items-start	Align items to start (both axes)
place-items-center	Center items both vertically and horizontally
items-center	Align items vertically
justify-center	Align items horizontally

6.6 Auto Columns/Rows

Class	Description
auto-cols-auto	Automatically sized columns
auto-rows-fr	Fractionally sized rows

7. Dark mode in Tailwind CSS:

Dark mode is a popular feature that changes the website's colors to be easier on the eyes in low-light environments. Tailwind CSS makes it easy to create a dark version of your site using a special dark: variant. For example, you can apply a light background with bg-white and a dark one with dark:bg-gray-800, and Tailwind will automatically choose the right style depending on the user's system preferences.

7.1 How It Works by Default:

By default, Tailwind uses the browser's built-in setting through prefers-color-scheme to detect if dark mode is enabled. You simply add classes like dark:bg-black or dark:text-white, and Tailwind will show those styles when the user has dark mode turned on in their operating system or browser settings.

7.2 Making Dark Mode Toggle Manually:

Instead of relying only on system settings, you can make dark mode work with a manual toggle using a dark class on the <html> tag. In Tailwind v4.1, you enable this behavior using the @custom-variant feature in your input.CSS file:

```
@custom-variant dark (&:where(.dark, .dark *));
```

Then, when the user clicks a "Dark Mode" button on your site, you can use JavaScript to add or remove the dark class on the <html> element, and even save their choice in localStorage.